



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(Национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ: ИУ «Информатика и системы управления»

КАФЕДРА: ИУ1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №5

«Нейросети для распознавания когнитивных состояний»

по дисциплине

«Эргатические системы»

Выполнила: Калюжная В.Р.

Группа: ИУ141М

Цель работы

Исследование когнитивных характеристик мозга по данным ЭЭГ с целью распознавания моторных образов (представления о движении) — классификация сжимания левого или правого кулака.

Ключевой навык

Применение алгоритмов классификации на основе нейронных сетей, включая этапы фильтрации, вейвлетпреобразования для визуализации временных рядов и построение 1DCNN (или гибридных архитектур) для бинарной классификации.

Ход работы

1. Загрузка датасета

Для выполнения работы используется набор данных EEG Motor Movement/Imagery Dataset (версия 1.0.0), доступный на PhysioNet .

Состав данных: Записи ЭЭГ у испытуемых, выполняющих задание по воображению движения (Motor Imagery — MI). В рамках данной задачи нас интересует различие между воображением сжатия левого и правого кулака.

Файлы:

`тренировочная выборка.csv` — многоканальный ЭЭГ-сигнал для обучения.

`тестовая выборка.csv` — данные для оценки качества модели.

`таргет тренировочной выборки.csv` — метки классов (0 — левый кулак, 1 — правый кулак).

`таргет тестовой выборки.csv` — эталонные метки для финальной проверки.

2. Анализ датасета и предобработка

ЭЭГ-сигнал характеризуется низким отношением сигнал/шум и нестационарностью. Для подготовки данных необходимо выполнить следующие шаги:

1. **Фильтрация:** Примените полосовой фильтр (например, Баттерворта 5го порядка) в диапазоне 8–30 Гц. Этот диапазон охватывает мюритм (8–12 Гц) и бетаритм (16–30 Гц), которые десинхронизируются (уменьшаются по амплитуде) при реальном или воображаемом движении, что является физиологической основой BCI .

2. **Вейвлетпреобразование (ВП):** В отличие от Fourier-анализа, ВП позволяет сохранить временную локализацию событий, что критично для анализа моторных образов, длящихся несколько секунд .

Метод: Используйте Continuous Wavelet Transform (CWT) с вейвлетом Morlet.

Результат: Преобразуйте одномерный временной сигнал каждого канала в двумерное изображение — скалограмму (Scalogram). По оси X будет время, по оси Y — масштаб (частота), а цветом отображается мощность сигнала .

Реализация: Это преобразование позволяет использовать ЭЭГ не просто как временной ряд, а как "картинку" (частотновременную карту), что иногда оправдывает применение 2D сверток, но в классическом пайплайне мы будем кормить сеть признаками, извлеченными из этих карт.

3. Проектирование архитектуры нейронной сети

Для классификации временных рядов (или их признаков) оптимальной является 1D Convolutional Neural Network (1DCNN). Она автоматически извлекает пространственновременные паттерны из сигнала или скалограммы.

Обоснование выбора 1DCNN:

Исследования показывают, что 1DCNN эффективна для двухклассовой классификации Motor Imagery, особенно в сочетании с вейвлетпреобразованием. Увеличение глубины сети (количества слоев) коррелирует с ростом точности, так как увеличивается количество обучаемых параметров, способных улавливать сложные зависимости .

Предлагаемая архитектура модели:

```
```python
```

Псевдоструктура модели (Keras/TensorFlow)

```
model = Sequential()
```

Входной слой: принимает обработанный сигнал или признаки скалограммы

Block 1

```
model.add(Conv1D(filters=32, kernel_size=5, activation='relu',
input_shape=(input_length, channels)))
model.add(BatchNormalization())
model.add(MaxPooling1D(pool_size=2))
model.add(Dropout(0.3))
```

Block 2 (увеличиваем глубину)

```
model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling1D(pool_size=2))
model.add(Dropout(0.3))
```

Block 3

```
model.add(Conv1D(filters=128, kernel_size=3, activation='relu'))
model.add(BatchNormalization())
model.add(GlobalAveragePooling1D())
```

 Альтернатива Flatten для уменьшения переобучения

```
Выходной блок (классификатор)
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))
```

 Бинарная классификация  
...

#### 4. Параметры обучения

Учитывая специфику задачи (физиологические сигналы подвержены вариативности), конфигурация обучения играет ключевую роль.

Функция потерь: `'binary_crossentropy'` — стандартный выбор для бинарной классификации. Альтернативные функции (например, `'logcosh'`) иногда показывают лучшую сходимость на зашумленных данных ЭЭГ, но `'binary_crossentropy'` является базовым стандартом.

Оптимизатор: Adam с обучением по умолчанию (`learning rate = 0.001`). Adam хорошо справляется с разреженными градиентами и зашумленными задачами, характерными для BCI.

Веса классов: `{0: 1.0, 1: 1.0}`. Хотя в вашем задании веса указаны как равные, рекомендуется проверить баланс классов в таргетах. При естественном дисбалансе (например, правшей больше), коррекция весов может повысить чувствительность (Sensitivity) модели.

Метрики:

Ассурасу — общая доля правильных ответов.

AUCROC — способность модели различать классы (порогонезависимая метрика).

Карра Score (Коэн) — учитывает случайные совпадения, что важно для физиологических данных с низким SNR.

#### 5. Ожидаемые результаты

На основе литературного обзора для схожих задач (PhysioNet, MI) ожидаются следующие результаты при использовании связки "Wavelet Transform + 1DCNN":

Точность (Ассурасу): на валидационной выборке ожидается в диапазоне 72% – 82%.

1DCNN малой глубины (2 слоя) показывает около 72%.

Современные архитектуры с Attention механизмами и MinMax нормализацией достигают 82% на тестовых данных.

Максимальные показатели в лабораторных условиях с жестким контролем данных достигают 96.68%, но для "сырых" данных и

типовой архитектуры 80% — это отличный подтвержденный результат .

#### 6. Заключение по этапам кода

При реализации в Python (например, в Jupyter Notebook) последовательность будет следующей:

1. Import: ``numpy``, ``pandas``, ``scipy.signal`` (фильтрация), ``pywt`` (вейвлеты), ``tensorflow``/``pytorch``.
2. EDA: Визуализируйте "сырые" сигналы для левого и правого кулака, чтобы убедиться в наличии EventRelated Desynchronization (ERD).
3. Препроцессинг: Напишите функцию ``preprocess_eeg(raw_data)``, которая применяет фильтр, нормализацию (``MinMaxScaler`` или ``Zscore``) и нарезку на эпохи .
4. Вейвлет: Для каждого образца примените ``pywt.cwt`` для получения скалограммы (или скормите сетке 1DCNN сырые отфильтрованные данные — второй подход быстрее).
5. Обучение: Скомпилируйте модель (``binary_crossentropy``, ``adam``), обучите 50100 эпох с Early Stopping.
6. Оценка: Постройте матрицу ошибок (Confusion Matrix) и кривую ROC.

